

```
path: ~/.npm
  key: deps-node-modules-${{ hashFiles('**/package-lock.json') }}
  - name: Install dependencies
    run: npm ci
  - name: Test code
    id: run-tests
    run: npm run test
  - name: Upload test report
    if: failure() && steps.run-tests.outcome == 'failure'
    uses: actions/upload-artifact@v3
    with:
      name: test-report
      path: test.json
build:
  needs: test
  runs-on: ubuntu-latest
  steps:
    - name: Get code
      uses: actions/checkout@v3
    - name: Cache dependencies
      id: cache
      uses: actions/cache@v3
```

Show less ^



Here is the complete GitHub Actions workflow with the missing `build` job filled in. It includes:

- Trigger on `push`
- `test` job
- Dependency caching
- Conditional test report upload on failure
- `build` job that depends on the `test` job
- Build artifact upload

</> YAML



name: Deployment Exercise 1

on:

push:

branches:

- main
- master

jobs:

test:

runs-on: ubuntu-latest

steps:

- name: Get code
uses: actions/checkout@v3
- name: Cache dependencies
id: cache
uses: actions/cache@v3
with:
 path: ~/.npm
 key: deps-node-modules-\${{ hashFiles('**/package-lock.json') }}
- name: Install dependencies
run: npm ci
- name: Test code
id: run-tests
run: npm run test
- name: Upload test report
if: failure() && steps.run-tests.outcome == 'failure'
uses: actions/upload-artifact@v3
with:
 name: test-report
 path: test.json

build:

needs: test

runs-on: ubuntu-latest

steps:

- name: Get code
uses: actions/checkout@v3
- name: Cache dependencies
id: cache
uses: actions/cache@v3
with:
 path: ~/.npm

```

      key: deps-node-modules-${ hashFiles('**/package-lock.json') }}

- name: Install dependencies
  run: npm ci

- name: Build website
  run: npm run build

- name: Upload build artifacts
  uses: actions/upload-artifact@v3
  with:
    name: build-files
    path: |
      dist
      build

```

Explanation

- `needs: test` ensures the `build` job only starts if the `test` job succeeds.
- `actions/cache@v3` caches the global npm cache (`~/.npm`) to speed up future `npm ci` installs.
- `failure() && steps.run-tests.outcome == 'failure'` uploads the test report only if the test step fails.
- `actions/upload-artifact@v3` stores the build output (`dist` or `build`) so it can be downloaded from the workflow run.

Note: Use `dist` for Angular, Vue, Vite, etc., and `build` for Create React App. Keep only the directory your project actually generates.



explains Upload test report section

Let's examine this step line by line:

</> YAML



```

- name: Upload test report
  if: failure() && steps.run-tests.outcome == 'failure'
  uses: actions/upload-artifact@v3
  with:

```

```
name: test-report
path: test.json
```

What does this step do?

This step uploads the file `test.json` as a GitHub Actions artifact **only when the tests fail**.

Artifacts are files that GitHub stores after a workflow run. You can download them later from the workflow page.

Line-by-line explanation

1. Step Name

</> YAML



```
- name: Upload test report
```

This is simply the name shown in the GitHub Actions UI.

Example:

```
✓ Get code
✓ Install dependencies
✗ Test code
✓ Upload test report
```



2. Conditional Execution

</> YAML



```
if: failure() && steps.run-tests.outcome == 'failure'
```

This is the most important part.

There are **two conditions**.

Condition 1

</> YAML



```
failure()
```

This is a GitHub Actions function.

It returns **true** if **any previous step in the current job has failed**.

Example:

Get code	✓
Install deps	✓
Test code	✗



Since one step failed,

```
failure() = true
```



If everything succeeds:

Get code	✓
Install deps	✓
Test code	✓



then

```
failure() = false
```



Condition 2

</> YAML



```
steps.run-tests.outcome == 'failure'
```

This checks the outcome of a **specific step**.

Remember this step:

</> YAML



```
- name: Test code  
  id: run-tests
```

```
run: npm run test
```

Because we gave it

```
</> YAML
```

```
id: run-tests
```



GitHub stores information about it.

We can access it like this:

```
</> YAML
```

```
steps.run-tests.outcome
```



Possible values are:

```
success  
failure  
cancelled  
skipped
```



If the tests fail:

```
steps.run-tests.outcome
```



becomes

```
failure
```



So

```
</> YAML
```

```
steps.run-tests.outcome == 'failure'
```



returns

```
true
```



Why check both?

Suppose another step failed.

Example:

```
Checkout      ✓  
Install deps  ✗  
Test code     (never ran)
```



Here

```
failure() = true
```



but

```
steps.run-tests.outcome = skipped
```



Therefore

```
steps.run-tests.outcome == 'failure'
```



is false.

Overall:

```
true && false  
=  
false
```



So the report is **not uploaded**, because the tests never actually ran.

This prevents uploading a report file that doesn't exist.

3. Upload Artifact Action

```
</> YAML
```



```
uses: actions/upload-artifact@v3
```

This is an official GitHub Action.

Its job is to save files generated during the workflow.

Think of it as:

"Store these files so I can download them later."

4. Artifact Name

</> YAML



name: test-report

This is the name displayed in GitHub.

Workflow page:

Artifacts



test-report

5. Path

</> YAML



path: test.json

This tells GitHub which file to upload.

Suppose your project generates

```
project/  
|  
├─ package.json  
├─ test.json  
└─ src/
```



Then GitHub uploads

test.json



You can also upload directories:

</> YAML




```
path: reports/
```

or multiple files:

```
</> YAML
```



```
path: |  
  reports/  
  coverage/
```

Example Workflow

Suppose

```
</> YAML
```



```
npm run test
```

runs Jest.

Result:

```
PASS Login Test  
PASS Register Test  
FAIL Payment Test
```



The test command creates

```
test.json
```



Now the workflow becomes

```
✓ Checkout  
✓ Install dependencies  
✗ Test code  
✓ Upload test report
```



GitHub stores

```
Artifacts  
└─ test-report  
    └─ test.json
```



You can download it from the workflow page to inspect the test results.

What if tests pass?

```
Checkout      ✓  
Install deps  ✓  
Test code     ✓
```



Then:

```
failure() = false
```



So

```
false && anything  
=  
false
```



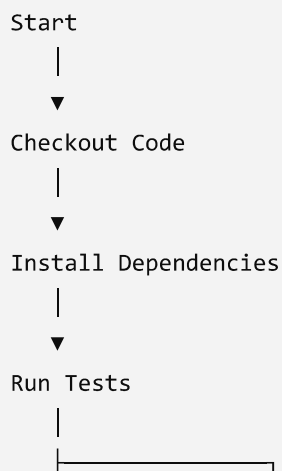
The upload step is skipped:

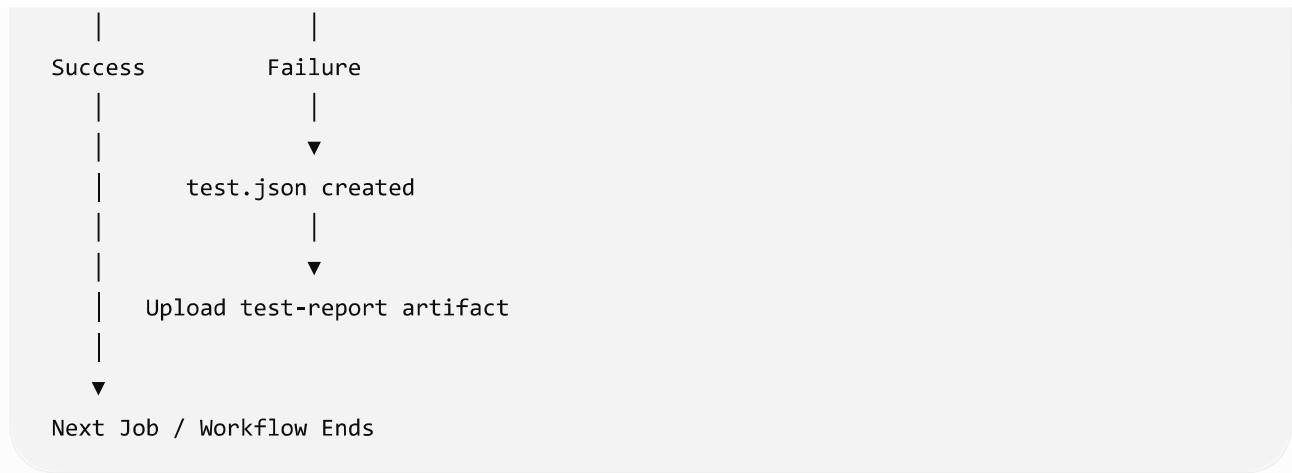
```
✓ Checkout  
✓ Install deps  
✓ Test code  
o Upload test report (Skipped)
```



This saves storage because there's no need to upload a failure report when the tests succeeded.

Execution Flow





This pattern is commonly used in CI pipelines because it preserves failure logs and reports for debugging while avoiding unnecessary artifact uploads when everything passes.



Do you like this personality?

